

Examples of Iterative Loops Patterns

Conversation Log | June 27, 2026

USER

give examples of Iterative loops patterns

CLAUDE

Here's one example for each of the three patterns under Iterative Loops — Reflexion, ReAct, and Test-Driven Prompting.

Reflexion

Generate an attempt, evaluate it against the actual goal or a concrete result (not just a gut check), feed the failure back in explicitly, and retry — the loop continues until the evaluation passes rather than stopping after one shot.

“Write a function in TextFinder1's core crate that strips ANSI color codes from a line before writing match output to a log file. Run it against this test line with embedded color codes, show me the actual output, and if it still contains escape sequences, fix it and try again — repeat until the test line comes out clean.”

First attempt might use a regex that catches `\x1b[31m` but misses `\x1b[0m` reset sequences with a different parameter count. The model runs the test, sees leftover escape bytes in the output, recognizes the regex's character class was too narrow, widens it, and re-runs — continuing until the test line is actually clean rather than assuming the first regex was right.

Why it matters: the loop is anchored to an observable result (the actual test output), not the model's confidence in its own code — so it keeps correcting until reality confirms it, not until the explanation merely sounds plausible.

ReAct (Reason + Act)

Interleave reasoning with actual tool calls or actions, observing the real result of each step before reasoning about the next one — rather than reasoning all the way through up front and only then acting.

“Find out why TextFinder1's build is failing on Windows. Don't guess at the whole fix up front — run the build, look at the actual error, reason about what it means, then take the next action based on that, and keep going until it builds clean.”

The model runs cargo build, sees a specific error about a crate version mismatch, reasons “this looks like a Cargo.lock conflict between RustCmdLine and the workspace root,” checks Cargo.toml, observes the actual version pins, decides to bump one dependency, rebuilds, observes a new error (now a missing feature flag), and continues — each action informed by the real output of the previous one rather than a plan made blind at the start.

Why it's distinct from Reflexion: Reflexion loops on a single artifact (write code, test it, fix it), while ReAct interleaves reasoning with arbitrary actions/tool use as it goes — the next action might be totally different in kind from the last one (read a file, then run a command, then edit a different file), not just a revision of the same thing.

Test-Driven Prompting

Write the tests or acceptance criteria first, then prompt for an implementation that must satisfy them — inverting the usual order so “done” is defined before any code exists, rather than checked afterward.

“Before writing the multi-extension matching logic for TextFinder1, write 4-5 test cases covering: single extension, comma-separated list, extension with/without leading dot, and an empty extension string. Show me the tests first. Only after I approve them, write the implementation that passes all of them.”

The model drafts tests like `assert_eq!(parse_extensions("rs,toml"), vec!["rs", "toml"])` and `assert!(parse_extensions("").is_err())` — and in writing them, surfaces a question you hadn't pinned down yet: should “.rs,toml” (mixed dot usage) be allowed, or rejected as malformed? You resolve that before any implementation exists, then the model writes code specifically to make the agreed-on tests pass.

Why it's distinct from the other two: Reflexion and ReAct both react to outcomes after an attempt; Test-Driven Prompting front-loads the definition of success so the very first implementation attempt is already aimed at a fixed target, rather than discovering the target through trial and error.

The throughline across all three: each one turns a single best-effort generation into a loop anchored to something checkable — a test result, a real tool observation, or a pre-agreed pass/fail criterion — so correctness is verified during the process instead of assumed at the end.